Landseer: Exploring the Machine Learning Defense Landscape

Abstract

Machine learning systems face diverse threats that undermine robustness, privacy, and fairness. Although many defenses have been proposed, each typically addresses a single risk in isolation. Real-world deployments, however, require these defenses to be composed to meet multiple guarantees simultaneously. The process of composing defenses is complex and not well understood, and its impact on performance and security remains unclear.

We present Landseer, a modular framework for integrating machine learning (ML) defenses into the ML lifecycle and systematically evaluating their composition. Landseer encapsulates defenses as containerized modules, allowing existing and new techniques to be plugged in with minimal effort. Its evaluation engine automates experiments across multiple metrics, supporting the study of defenses both individually and in combination. In a preliminary study, we identified 35 state-of-the-art machine learning defenses. After filtering for reproducibility, we analyzed their performance using Landseer's unified evaluation process.

Our findings reveal gaps in replicability across defense families and provide insights into the challenges and opportunities in integrating multiple defenses, establishing a foundation for improving the reliability of machine learning systems.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **Security and privacy** → *Systems security*.

Keywords

Trustworthy Machine Learning, Defenses, ML Framework

ACM Reference Format:

. 2018. Landseer: Exploring the Machine Learning Defense Landscape. In *Proceedings of ACM SIGSAC Conference on Computer and Communications Security (CCS '26).* ACM, New York, NY, USA, 17 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The increasing popularity of Machine Learning (ML) systems in critical applications and industries such as cybersecurity [5, 8], healthcare [12, 45, 77], and finance [86] has raised growing concerns about the susceptibility of these systems to various types of attacks and trust concerns. Notable incidents include physical evasion of traffic sign detectors using small stickers [30], adversarial perturbations that can alter medical image diagnoses [97], and data poisoning attacks that lead to backdoors [112]. These threats target various trustworthy ML dimensions including performance [47, 100], robustness [31, 67], privacy [20, 32, 54, 62, 84, 107], and fairness [49, 78, 99], among others.

In response, industry and academia have developed defense techniques [23, 65, 72] to prevent attacks targeting any of the trustworthy machine learning qualities described above. For example, robustness risks can be addressed by adversarial training [110, 114], and backdoors can be mitigated via data sanitization [11]. Each defense is typically designed to be applied at a specific stage of the ML lifecycle and can serve as a building block for constructing secure ML systems. In real-world scenarios, multiple risks or attack vectors are often present simultaneously. As a result, it may be necessary to incorporate various defense strategies to address a wide range of security and reliability requirements. Consequently, we need the ability to combine defenses while keeping their purpose and effectiveness intact.

Unfortunately, combining multiple ML defenses is not trivial. Composability of defenses is not well understood, and the aggregate effect on system performance and security guarantees is difficult to predict. This difficulty is due to the diversity of existing techniques. For any one risk class, a variety of defenses exist that are applied at different stages of development or deployment and are configured by a unique set of parameters. As a result, the landscape of possible defenses that could be applied to an ML system grows with combinatorial complexity, making the impact of combining multiple trustworthy machine learning defenses challenging to evaluate.

Due to the aforementioned challenges, prior efforts to combine ML defenses and study their interactions and effects on system performance and risks have been mainly limited to pairwise evaluations consisting of two ML defenses at a time [19, 28, 33, 68, 92, 96]. Duddu et al. [28] studied the interactions between defenses and risks, examining the unintended effect of one defense on other risks (not targeted by the defense). However, this work does not offer any insight into how defenses interact with each other. In a follow-up, Duddu et al. [29] proposed a non-empirical model to predict if combining specific existing defenses will be effective. However, the model is primarily proposed for pairwise combinations, which limits expandability when multiple defenses are considered in the same or different stages of the pipeline.

In this paper, we close this gap by developing Landseer, a comprehensive framework to integrate and systematically analyze ML defenses in a typical ML pipeline to discover automated insights on defense interactions. We define and explore the notion of *composability* for ML defenses, investigating structural requirements for a defense to coexist with other tools in the pipeline, as well as qualitative interference between defenses that affect trust metrics. Landseer's architecture consists of an onboarding module, a pipeline explorer, and an interference analysis engine. The onboarding module takes a tool description and corresponding software artifact and generates a reproducibility record together with a structurally composable artifact. The pipeline explorer is the core empirical component that collects data on qualitative composability of defense combinations. The interference analysis engine performs

CCS '26,

ACM ISBN 978-1-4503-XXXX-X/XXXX/XX
https://doi.org/XXXXXXX.XXXXXXX

root-cause analysis on collected data to identify sources of interference and provide additional insights about the tools and their interactions.

We demonstrate the capabilities of LANDSEER by surveying 35 defense techniques from literature and examining their composability. Based on this survey, we identify integration barriers and compile guidelines for researchers to avoid composability obstacles. From the examined tools, we onboard 11 and demonstrate that LANDSEER is able to reproduce existing results on interactions of defenses, and can further surface new insights. Notably, our results show that defense interactions depend on several factors, including the stage at which a defense is applied and the order of defenses within a stage, which have not been examined in prior work. LANDSEER can help ML practitioners identify best-performing combinations according to a security policy, and enable ML researchers to extract new insights into ML defense interactions to build easy-to-integrate defenses and lower barriers for their adoption.

Our contributions can be summarized as follows:

- **Define and study the notion of composability** of various Trustworthy Machine Learning (TML) techniques in a Machine Learning Pipeline. Our taxonomy outlines two composability criteria, structural and qualitative, that describe a defense's ability to be integrated in an ML deployment.
- **Design and implement LANDSEER**, a generalizable framework to adopt and analyze the composability of defenses.
- **Test a corpus of 35 TML techniques** against LANDSEER to replicate previous literature on TML defense interactions, as well as discover new adverse interactions (or interferences) among tools across 700 total combinations.
- **Identify new insights to aid in tool composability** through two case studies to help practitioners and researchers in future tool development.

2 Background

2.1 Machine Learning Operations

Machine learning operations (MLOps) describe the activities and lifecycle to create and deploy machine learning models. Example operations include collection and processing of data, the use of this data to train a model, and deployment of the model to users. At a high level, we can categorize the MLOps lifecycle into the following stages: (i) *data processing* involves collecting a dataset and processing it through cleaning and transformation operations, (ii) *training* consists of designing and training an ML model over the collected training data, (iii) *testing* carries out the evaluation of the resulting model from the previous stage on a holdout set of data across a set of metrics and might involve further adjustments to the model, and (iv) *deployment* where the model is deployed at scale and performs computations over production data.

2.2 Trustworthy Machine Learning Techniques

ML systems are designed to perform well (for example, have high prediction accuracy) on never-before-seen data. However, ML systems should also exhibit other desirable characteristics for deployment in the real world, such as robustness, fairness, and privacy protection. We provide an overview of Trustworthy Machine Learning (TML) techniques that we study in this work, and present relevant definitions and evaluation metrics summarized in Table 1.

Table 1: Description of ML risks and corresponding TML techniques and metrics for evaluating the effectiveness of each technique. ↑ (↓) indicates a higher (lower) value is better.

Risk	TML Technique	Eval. Metric
Evasion	Adv. Robustness $\mathbf{T}^{\mathbf{ev}}$	Adv. robustness acc. $\mathbf{m}^{\mathbf{ev}}(\uparrow)$
Poisoning	Outl. Robustness $\mathbf{T}^{\mathbf{ou}}$	Att. success rate $\mathbf{m}^{\mathbf{ou}}(\downarrow)$, area under ROC curve $\mathbf{m}^{\mathbf{ar}}(\uparrow)$
Privacy	Diff. privacy $\mathbf{T}^{\mathbf{dp}}$	DP parameters $\mathbf{m}^{\mathbf{dp}}_{\epsilon}(\downarrow)$ and $\mathbf{m}^{\mathbf{dp}}_{\mathbf{mia}}(\downarrow)$
Intel. Prop.	Watermarking $\mathbf{T}^{\mathbf{wm}}$	Watermark detection accuracy $\mathbf{m}^{\mathbf{wm}}(\uparrow)$
Intel. Prop.	Fingerprinting $\mathbf{T}^{\mathbf{fp}}$	Fingerprint detection confidence score $\mathbf{m}^{\mathbf{fp}}(\uparrow)$
Bias	Group Fairness $\mathbf{T}^{\mathbf{gf}}$	Demographic parity $\mathbf{m}^{\mathbf{fa}}(\uparrow)$
Transparency	Explanation $\mathbf{T}^{\mathbf{ex}}$	Explanation error $\mathbf{m}^{\mathbf{ex}}(\uparrow)$

Adversarial Robustness. An adversarial evasion attack deceives the model into making incorrect predictions by adding carefully crafted perturbations to input [34, 91]. Evasion robustness techniques $\mathbf{T}^{\mathbf{ev}}$ aim to suppress the effects of adversarial examples by improving model resilience. Methods to achieve robustness include adversarial training [59, 83], randomized smoothing [24], and input data transformations to strip perturbations [23, 36, 102]. Evaluating the robustness of a model can be performed by testing the model against a dataset containing adversarial examples. We use the adversarial robustness accuracy $\mathbf{m}^{\mathbf{ev}}$ as the evaluation metric measuring the percentage of adversarial examples correctly classified.

Outlier Robustness. Outliers are data points (accidentally or maliciously added) that deviate from the distribution of other points. Techniques to improve outlier robustness $\mathbf{T}^{\mathbf{ou}}$ include dataset sanitization [11, 37, 87] and model-focused purification [26, 51, 101]. Depending on the technique, we use attack success rate $\mathbf{m}^{\mathbf{ou}}_{\mathbf{asr}}$ [10] or the area under the receiver operating characteristic curve $\mathbf{m}^{\mathbf{ou}}_{\mathbf{ar}}$ [25] as metrics to evaluate robustness to outliers.

Privacy. Privacy attacks include membership [84], attribute [61], and reconstruction [15] attacks, which aim to leak the usage of a data point or its hidden attributes or recreate them from the model's output. Differential Privacy (DP) is a popular technique $\mathbf{T}^{\mathbf{dp}}$ to protect privacy by adding controlled yet random noise to the data while enabling the learning task simultaneously. DP mechanisms can be designed and evaluated based on the privacy budget $\mathbf{m}^{\mathbf{dp}}_{\epsilon}$ (indicates the strength of the privacy guarantee) , and $\mathbf{m}^{\mathbf{dp}}_{\mathbf{mia}}$ [1, 65] (quantifying susceptibility to membership inference attacks).

Watermarking. Adversaries may attempt to steal ML models to avoid the high training or continuous per query costs [69, 95]. Watermarking techniques $\mathbf{T}^{\mathbf{wm}}$ embed ownership information into trained models that can be later verified via trigger/backdoor marks using watermarked training data or after-training finetuning [2],

training-time embeddings [98], API-time dynamic watermarking [93], and data-centric tracing [82]. The watermark accuracy metric $\mathbf{m^{wm}}$ evaluates the presence of watermarks in a model and the percentage (or fraction) of the correctly identified watermarks.

Fingerprinting. Fingerprinting is a technique $\mathbf{T^{fp}}$ to verify the ownership of an ML model by analyzing inherent characteristics such as the model's decision-boundary signature [74]. To compute the accuracy of a fingerprint, we can use a confidence score metric $\mathbf{m^{fp}}$ accompanying a prediction that a particular model contains a specific fingerprint [14].

Fairness. ML models are susceptible to biases that would make their decisions *unfair* [6, 9]. In this paper, we consider the notion of group fairness, which ensures that different groups are treated equally [75]. Techniques for group fairness ($\mathbf{T^{fa}}$) include rebalancing training data [27] or modifying the learning objective [3] To evaluate fairness, we adopt the demographic parity (statistical parity) metric $\mathbf{m^{fa}}$ to measure whether different groups receive positive outcomes at equal rates.

Explanation Understanding model decisions can help promote trust and fairness [53]. Explanation techniques $\mathbf{T^{ex}}$ aim to clarify how predictions are made by methods including examining how changes during training data affect outputs, or incorporating explanation terms directly into the model's objective function [57, 79]. We adopt the explanation error metric $\mathbf{m^{ex}}$ which quantifies how accurately the explanation reflects the model's true behavior [76].

3 Landseer Overview

Thus, Landseer is a *framework* designed to explore the ways in which TML defenses tools can be composed. To do so, it relies on modules to perform comprehensive composability tests on available TML defenses. This section presents the definitions required to understand the framework, the flow users carry out to use it, and its three major building blocks.

3.1 Definitions

We begin by defining the following concepts:

- **Structural Composability** specifies whether a tool can be integrated into a pipeline and produce an output, i.e, whether its inputs and outputs are compatible with adjacent components such that it can be seamlessly chained within a larger workflow.
- **Qualitative Composability** describes whether two or more defenses can coexist without significantly degrading overall system or defense performance.
- **TML defenses Interference** occurs when one tool changes the performance of another. When two tools are not qualitatively composable, they negatively interfere. To provide a deeper insight into how tools can fail to compose, Landseer also identifies *interference patterns* (or types), which can ultimately assist practitioners in communicating the consequences of deploying TML defenses together.

We will utilize these definitions throughout the rest of the paper to characterize how TML defenses may (or may not) integrate.

3.2 Landseer Flow

A user of Landseer, such as an ML engineer, is tasked with ensuring TML defenses can be successfully integrated into their system. To do so, they must first identify target TML defenses tools denoted as a tuple of $< Code, Target >$. $Code$ refers to an existing implementation of such a tool, whereas $Target$ is a vector containing a specific benchmark for TML defenses metrics. We provide some examples for this in Section 6.

Besides a collection of tools to test and integrate, a user must also provide a model M, a dataset D, and a hyperparameter set H. In doing so, a user can verify whether a tool is composable in their particular setting.

3.3 Landseer Components

Landseer consists of three major components:

- **Onboarding Module:** This module is tasked with converting TML defenses into composable artifacts. To do so, it requires a tool description (e.g, an academic paper) and its corresponding software artifact. From these, it builds a reproducibility record (i.e., their respective expected reproducibility values), runs a series of tests to confirm reproducibility and structural composability, and produces an artifact that can be integrated into an MLOps pipeline.
- **Pipeline Explorer:** After the tool(s) pass the onboarding module, the Pipeline Explorer module builds combinations to identify whether tools produce satisfactory results (i.e., are qualitatively composable, as described below). The pipeline explorer also verifies whether TML defenses defenses work within a target dataset, model-architecture, and other user-defined parameters. At a high level, this process can be an enumeration (or cross-product) of all available tools for a particular configuration, though Landseer applies various optimizations to avoid recomputing intermediate artifacts.
- **Interference Analysis:** Once all pipeline combinations are explored, the results are processed through an analysis engine, which provides feedback on which tool combinations are interfering, the type of interference, and additional insights about the tools.

Figure 1 depicts these modules and the flow. We will explain these components in detail in Section 4.

3.4 Threat Model

Landseer is at its core a meta-framework for trustworthy machine learning composition. As such, it inherits attacker goals and capabilities from the defenses integrated in our study. Further, Landseer explicitly operates under a framework placing defense benchmarks against defense constructions, which directly encapsulate the relationship between attackers and defenders for a particular Trustworthy Machine Learning setting. Lastly, we assume there are no emergent attack classes derived from threat composition that are not addressed by composing their corresponding defenses.

The security of the framework itself falls within standard distributed system assumptions under a non-byzantine setting. Operators of a Landseer deployment are in full control of all the nodes in the infrastructure, and full compromise of a particular node will result in faulty computation. Attacks that result in compromised

input tooling (e.g., supply chain attacks through research code) are addressed under regular risk assessment methodologies and are out of scope for this paper.

4 Landseer Architecture

In this section, we further develop the specific modules that are part of the Landseer framework.

4.1 Onboarding Module

Before a TML tool can be used in a Landseer pipeline, it must first be converted from a standalone research artifact into a reliable pipeline component. This step is necessary because TML tools are often released with different assumptions about datasets, model architectures, dependencies, input formats, output formats, and evaluation scripts. Moreover, a tool may run correctly in its original setting (e.g., as a standalone Python script), but still be difficult to compose with other tools in an MLOps pipeline [13] (Refer Table 5). This is important because structural composability forms the baseline for qualitative composability – Landseer can only evaluate how tools interact if they can be integrated into a pipeline.

In order to successfully adapt a TML tool into a pipeline, Landseer must ensure that the following invariants hold: In particular, it should determine: (i) whether the tool can be executed, (ii) whether it reproduces the behavior reported by the original artifact (e.g., it matches the results reported in the original paper), (iii) where the tool belongs in the ML lifecycle, and (iv) what input and output artifacts are required to chain it with other tools.

4.1.1 Reproduction Records. In order to track progress in adapting TML tools into the pipeline, Landseer keeps a *Reproduction Record* for each candidate tool. Succinctly put, this record stores both the evidence that the tool works in isolation and the interface information needed to use it inside a larger MLOps pipeline. A Reproduction Record contains three classes of information: artifact metadata, reproducibility outcome, and structural properties.

Artifact metadata. This includes the paper or artifact source, the code URL, the defense category, the intended pipeline stage, the supported datasets, the framework requirements, and the reported target metrics. These fields describe what the tool claims to do and where it is expected to fit in the trustworthy ML lifecycle. Populating this information is a structured process that can be carried out by a human or an automated agent.

Reproducibility Outcome. This includes whether the artifact executes successfully, and what target metric was reproduced. This part of the record establishes the standalone baseline for the tool. Without this baseline, it is difficult to determine later whether a change in performance is caused by the composition with another tool or by an already unstable artifact.

Structural Interface. This includes the expected input artifact, the produced output artifact, the required auxiliary files, the parameter schema, and any format constraints on datasets, model checkpoints, or framework versions. These fields determine the tool's structural composability: whether another stage of the pipeline can consume its outputs without any transformation (e.g., between data representations). This stage also identifies relevant configuration parameters (e.g., model architecture or dataset inputs) that may be needed to allow for extensibility.

4.1.2 Onboarding Funnel. Using this record, candidate tools can be "funneled" and promoted through three stages: reproducibility, structural compatibility, and containerization.

Reproducibility check. This check determines whether the tool can be executed and whether it matches the target behavior reported by the original artifact. The candidate tool is run in an environment similar to the original reported setup (like runtime, software dependencies), using the same dataset, model, parameters, and evaluation metric. If the artifact fails due to minor dependencies or compatibility issues, the tool may still be considered replicable if minimal changes allow it to run without altering the core method [110] (Refer to Table 5).

Structural check. If a tool is reproduced, the next phase determines its placement in the MLOps pipeline, what artifacts it consumes and produces according to the categories described in Section 4.2. This check records whether the tool is structurally composable in an MLOps pipeline. A tool may be composable as-is, may require a wrapper, or may be deemed non-composable. For instance, a tool that outputs outlier labels may become composable after a wrapper converts those labels into a filtered dataset that can then be transferred to other stages [113]. In contrast, a tool that only reports a final score without producing a reusable artifact may be reproducible but not structurally composable[13] (Table 5).

The structural check uses the input/output of a particular tool to establish its relative positioning in the MLOps pipeline. Landseer identifies the following TML tool types:

- **Pre-Training** techniques (T_{pre}) involve approaches that are applied before model training begins. Examples include dataset-level strategies that modify or transform the training data or methods that influence model architecture selection. The input and output here are datasets.
- **During-Training** techniques (T_{in}) intervene directly in the learning algorithm. Examples include modifying the objective function, optimization process, model initialization, or gradient computations. Input is a dataset and model architecture, and output is a trained model.
- **Post-Training** techniques (T_{post}) are applied on a pre-trained model. While they do not modify the original training procedure, they still affect the internal state of the model. Examples include fine-tuning, weight pruning, or additional calibration steps. Input is a trained model and a dataset, and output is a modified model.
- **Deployment** techniques (T_{dep}) are applied on the inputs to the model or its outputs during inference. Examples include input filtering to detect or block malicious queries, and output post-processing , and analyzing the model's internal activations or behavior at inference time. Input is a model and a dataset, and output is a dataset.

Containerization. This check determines whether the tool can be executed as an independent unit. Each tool passing the structural check is encapsulated in a standalone OCI [71] image that bundles its dependencies and exposes a standard execution interface. The

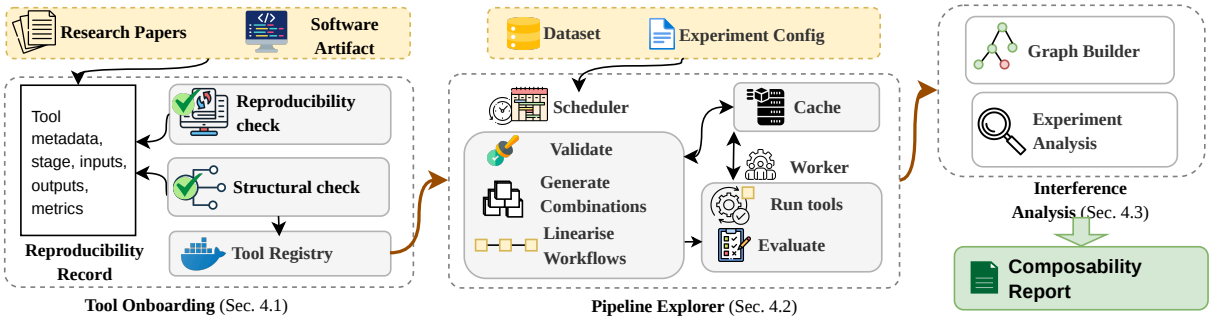

Figure 1: Overview of LANDSEER design. Practitioners define tools of interest in a domain-specific language, while machine learning researchers can integrate attacks and defenses to test against (Section 6.2). Results are produced as series of n-tuples describing a defense combination, a test accuracy, and a collection of TML defenses scores (Section 4).

containerization phase also embeds the reproducibility record to aid Pipeline Explorer when scheduling experiments.

4.2 Pipeline Explorer

Once tools are onboarded, the Pipeline Explorer is responsible for systematically evaluating how they behave in combination. It defines the experiment space, generates all valid defense compositions, schedules their execution, and coordinates between different runs.

The LANDSEER API utilizes two concepts to explore the composition space. First, an **Experiment**, which is defined by a model, a dataset, and a set of candidate defense tools for each stage of the ML pipeline. An experiment is a systematic exploration of all valid combinations of these tools, examining their collective effect on the model's utility and various TML evaluation metrics, such as those listed in Table 1. The results of all combinations are analyzed together to understand how different defenses interact and compare.

For each experiment, a series of **Combinations** are generated. A combination is one specific combination of defense tools applied to a given model and dataset. It represents a single path through the experiment's search space, executing the selected tools in sequence across the ML stages and producing a set of evaluation metrics.

4.2.1 Scheduler and Execution. To carry out an experiment, one Scheduler and a series of Worker nodes (described below) operate in conjunction to carry out the following passes.

The scheduler proceeds in three steps. First, it *validates* each tool against the experiment configuration, checking that the tool supports the requested dataset and that its input and output artifacts are compatible with adjacent stages. Second, it *generates* all valid pipelines from the remaining tools, producing the full set of stage-wise ordered combinations. Third, it *linearizes* each pipeline into an ordered sequence of atomic tasks suitable for distributed execution.

Scale and re-runs. Because many pipelines share common prefixes, naive re-execution is wasteful. The scheduler avoids redundant computation by assigning each task a content-derived identifier based on its tool, configuration, and input artifacts. If a matching result already exists in the cache, the task is skipped and its output is passed directly to the next stage. This deduplication significantly reduces total compute, particularly for Pre-Training and During-Training tasks that are shared across many pipelines. To support statistical robustness, the scheduler also allows pipelines to be re-run a configurable number of times with different random seeds, aggregating results across runs.

Workers. These are the execution units that carry out individual tasks. The scheduler assigns tasks to workers based on their resources (e.g., available GPUs), and workers execute tasks by invoking the corresponding TML tool with the appropriate inputs.

4.3 Interference Analysis

The results generated by LANDSEER (in Figure 1) can be used to identify sources of interference. LANDSEER presents these results as a vector (C, M), where C is a pipeline configuration (i.e., combination of tools) and $M = \{Acc, m_1, ..., m_n)$ is a list of performance criteria. Specifically, Acc is the accuracy on clean test set, and $\{m_1, \cdots, m_n\}$ are n trustworthy ML evaluation metrics.

We use performance metrics M to assess qualitative composability of defenses. LANDSEER performs a root-cause analysis to identify the sources of interference (e.g., settings or techniques that trigger interference) within a flagged combination. Later, we introduce different categories of interference and present examples of defense combinations for each type.

Given that LANDSEER generates a large quantity of combinations (our current corpus of tools generates more than 700 different combinations) we must systematically identify only the ones that are indicative of interference. To do so, we employ three steps for our data analysis pipeline: (1) build a data structure, namely an interference graph, that allows us to place any combination in relationship with other combinations, (2) threshold interference by an adjustable value relative to a baseline measure, and (3) classify interference patterns by inspecting the specifics between combination pairs and, optionally, their respective neighbors in the graph.

4.3.1 Interference Graphs. We start by constructing *interference graphs* for each defense tool (i.e., "focus tool"). The nodes in the interference graph represent a combination of defense tools in the pipeline, with the root node consisting of a baseline combination $\mathbf{c}_b$ with only the focus tool (and noop defenses in other stages). Similarly, other nodes in this inference graph will include the focus tool in combination with other defenses. Structurally, the interference graph holds a series of edges to represent *minimum distances* between different combination nodes. We describe graph edges as:

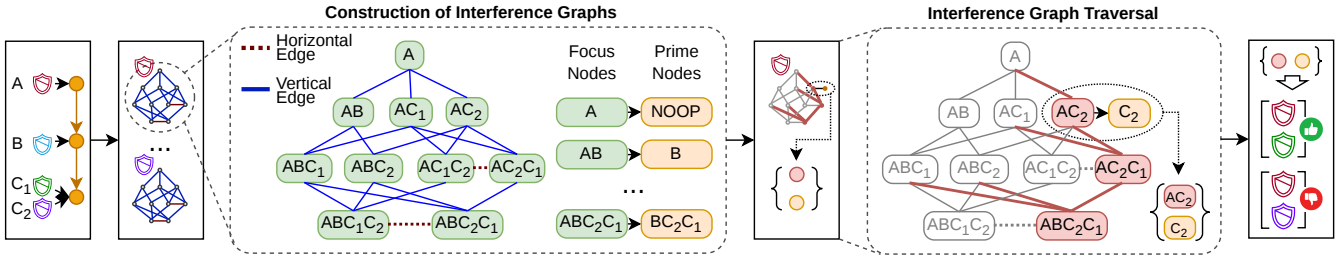

Figure 2: Results generation and analysis in LANDSEER. The pipeline result, ref Fig 1, is used as the input to the analysis algorithm, where comprehensive relationship graphs are built for each tool in the graph builder. A zoomed-in view of a sample comprehensive graph is seen in the Detailed Graph block. Each graph is passed through a traversal function, which draws different interference results.

- Vertical edges: We depict incremental cardinality relationships between the number of tools in each combination (node) with a vertical edge. Child nodes add exactly one tool to the parent combination in the corresponding stage. Nodes that appear as parents for multiple top nodes are represented only once, with multiple vertical edges pointing to them to ensures a connected, non-redundant representation of all possible incremental tool combinations.
- Horizontal edges: We define a neighboring combination as one in which the same tools are present but in different ordering, and depict neighboring combinations with a horizontal edge. Keeping Horizontal relationships aid in identifying ordering-based interference as we discuss later.

Algorithm 1 describes the construction of N graphs (one for each focus tool) from the set of all combinations C obtained from LANDSEER For each tool, we start from the lowest cardinality ($i = 1$ in line 5) and extract the combination with only the focus tool t (and noop in other stages) as the root node of the graph. In each following step, we extract nodes with cardinality i which include tool t (*FocusNodes* in line 6) and add them to the graph. For each node, we also extract and save the *prime* (*PrimeNodes* in line 7) which has the same set of tools but without the focus tool. This focus-prime relationship is used for our interference analysis as we detail later. For each node in the graph, we identify parent nodes (`Parent` function in line 12) and neighbor nodes (`Neighbor` function in line 17) and add vertical and horizontal edges respectively to the graph. As previously described, vertical edges connect parent nodes to child nodes which include all parent tools plus exactly one additional tool, preserving the order of tool application. Horizontal edges are added between nodes at the same cardinality level with the same tools in the combination but in a different order. This process continues until the graph G_t for focus tool t is completed. The previous steps are then repeated for the next focus tool until all N graphs, namely the set of interference graphs $\mathcal{G}$ are constructed.

Figure 2 depicts the construction of interference graphs for a pipeline with three stages with tools A ("Pre-Training" stage), B ("During-Training" stage), and $C1$, $C2$ ("Post-Training" stage). The graph construction shows the resulting interference graph for tool A as the focus tool. At each level of the tree, the child nodes are combinations with exactly one additional tool compared to their parents applied to its corresponding stage. Vertical edges are represented by blue solid lines and horizontal edges are represented by dotted red lines. For example, the line between nodes AC_2C_1 and ABC_2C_1 represent a parent-child relationship with tool B added to the child node, whereas the line between nodes AC_1C_2 and AC_2C_1 represent a neighbor relationship with the order of tools C_1 and C_2 are reversed in the two combinations.

After constructing the interference graphs, we devise a technique to perform root-cause analysis of interference. As previously described in Section 3.1, we want to flag combinations where the addition of a focus tool degrades (or improves) performance across one of the metrics in M compared to a baseline combination without

Algorithm 1: Construction of Interference Graphs

Input: Set of all combinations C, number of tools N
Result: Set of interference graphs $\mathcal{G}$

1 **for** $t = 1$ *to* N **do**
2 Initialize empty graph $G_t \leftarrow \{\}$
3 Initialize empty set $PrevNodes \leftarrow \{\}$
4 $k_t \leftarrow$ `MaxCardinality`(C, t)
5 **for** $i = 1$ *to* k_t **do**
6 $FocusNodes \leftarrow$ `ExtractCombs`(C, i, t)
7 $PrimeNodes \leftarrow$ `ExtractPrime`$(C, FocusNodes)$
8 G_t.`Append`$(FocusNodes)$
9 G_t.`Append`$(PrimeNodes)$
10 **if** $i > 1$ **then**
11 **for** (n_1, n_2) *in* $(PrevNodes, FocusNodes)$ **do**
12 **if** `Parent`(n_1, n_2) **then**
13 G_t.`AddVerticalEdge`(n_1, n_2)
14 **end**
15 **end**
16 **for** (n_1, n_2) *in* $(FocusNodes, FocusNodes)$ **do**
17 **if** `Neighbor`(n_1, n_2) **then**
18 G_t.`AddHorizontalEdge`(n_1, n_2)
19 **end**
20 **end**
21 **end**
22 $PrevNodes \leftarrow FocusNodes$
23 **end**
24 Add G_t to $\mathcal{G}$
25 **end**

Algorithm 2: Interference Graph Traversal

Input: Interference graph G_t
Result: Set of identified combinations $\mathcal{S}$

```
1  Initialize empty set S ← {}
2  Initialize empty set NodeList ← {}
3  Nodes ← ExtractLeaves(G_t)
4  NodeList.PushNode(Nodes)
5  while NodeList not empty do
6      Node ← NodeList.PopNode()
7      PrimeNode ← ExtractPrime(G_t, Node)
8      NInt ← Interference(Node, PrimeNode)
9      if NInt then
10         RootCause ← True
11         ParentNodes ← ExtractParents(G_t, Node)
12         for PNode in ParentNodes do
13             PrimeParent ← ExtractPrime(G_t, PNode)
14             PNInt ← Interference(PNode, PrimeParent)
15             if PNInt then
16                 NodeList.PushNode(PNode)
17                 RootCause ← False
18             end
19         end
20         if RootCause then
21             Neighbors ← ExtractNeighbors(G_t, Node)
22             S.Add([Node, PrimeNode, Neighbors])
23         end
24     end
25 end
```

the focus tool (prime combination). Our root-cause analysis identifies combinations with interference that: (1) reduce (or increase) one or more of performance metrics in M beyond a set threshold, and (2) has the lowest cardinality that introduces such a drop. This definition of root-cause combination ensures that the first instance of interference is identified by finding the lowers cardinality, and the interference is associated with the presence of the focus tool by comparing performance results to the prime node.

Our root-cause analysis is based on a graph traversal described in Algorithm 2 on the interference graphs constructed for each tool. Starting from the highest cardinality nodes in the graph (line 3), we identify nodes with interference by comparing the performance metrics to the prime combination (`Interference` function in line 8). Any node with interference that has a parent node with interference is not considered root-cause, and the parent node is instead added for analysis next (lines 12-19). The identified root-cause nodes together with their prime and neighbor nodes are then submitted to a characterization function described in Section 4.3.2.

Figure 2 illustrates the root-cause analysis steps through interference graph traversal. In this example, node AC_2 is identified as root-cause presenting interference while parent node A and prime node C_2 do not show the same interference.

Threshold Selection To prevent random noise from affecting our analysis, we define threshold values and mark interference if the metric values change beyond these thresholds. We define two thresholds for each metric, namely t_l and t_h. Changes to metric values below t_l is classified as negligible, between t_l and t_h is classified as moderate, and above t_h is classified as severe. We specify the values for t_l and t_h for our experiments in Section 6.1. This approach allows Landseer to classify the magnitude of metric changes, as well as the direction of the effect (positive, negative, or mixed), providing a standardized basis to evaluate when tool combinations meaningfully impact performance.

4.3.2 Interference Characterization. Having identified two *Comb* nodes that are root-cause and above a threshold, we can characterize the type of interference. Intuitively, to characterize the type of interference, we label the combination by three properties:(1) Cardinality difference between the nodes, as some sources of interference will appear on both vertical and horizontal edges (e.g., are the nodes on the same level or different levels?) (2) Structural difference between the *Comb* nodes (e.g., was there a change in ordering? was a new tool added in a particular stage of the pipeline?) (3) Specific effect caused (e.g., did it affect accuracy? or a single TMLD value?)

This characterization can allow us to empirically label and categorize types of interference, under the following types:

- **Global Interference (GI)** indicates interference of a tool on others' performance metrics M whenever it is applied in the combination.
- **Ordering Interference (OI)** represents interference between a set of tools only when a specific ordering of the tools is used.
- **Pairwise Interference (PW)** captures interference between exactly two tools as the change in their performance when used together vs. independently.

We will use this classification in Section 6.3.

5 Implementation

Beyond the architectural properties described above, Landseer's implementation required to overcome a series of challenges, primarily related to scale and combination pipelining.

Challenge 1: Tool Dependency Management. Defenses within Landseer employ a variety of machine learning frameworks, Python versions, and CUDA toolkits. Artifacts identified frequently rely on a diverse set of dependency stacks, rendering a unified shared runtime impractical. Landseer encapsulates each defense in a self-contained runtime image with fixed dependencies and versioned environment specifications identified during onboarding.

Workers select and are selected from the available and supported backends on the host platform (e.g., Docker [64] for container-enabled systems or Apptainer/Singularity[7] for restricted HPC environments). The runtime contract remains consistent across different backends, ensuring the same mounted inputs, configuration interface, and output directory structure are utilized.

Challenge 2: Artifact Heterogeneity Across Pipeline Stages. Much like dependencies, intermediate artifacts use a variety of bespoke formats. Every defense is scheduled with fixed mount points: `/data` (outputs from all upstream tasks), `/config` (model architecture and dataset parameters), and `/output` (where the tool writes its results). Landseer utilizes data stored in the reproduction record

to identify and inject any necessary data representation transformation steps. This may introduce concerns about the impact of such tasks, however, the onboarding process verifies reproducibility independently (i.e., outisde of the framework) and in a standalone pipeline inside of the framework that must match the target values in the reproduction record.

Challenge 3: Exploring the Combinatorial Space. Running each workflow independently is computationally expensive because many workflows share common prefixes. For instance, workflows that start with the same preprocessing defense applied to the same dataset perform identical early-stage computations.

Landseer compiles all valid combinations into a single global directed acyclic graph (DAG). Then, duplicate sub-paths are eliminated to avoid recomputation. Task equivalence is determined based on the tool used, combination configurations, and input artifact signatures. The scheduler processes the resulting graph in dependency order and prioritizes tasks likely to be reused. This approach enables shared prefixes to be computed once and reused across multiple downstream workflows, significantly improving computational efficiency.

Challenge 4: Intermediate Artifact Caching and Data Management. Landseer employs a two-level artifact caching system. The first level consists of a local disk cache for each worker, enabling direct retrieval of artifacts previously produced on the same machine. The second level uses a shared MinIO [94] object store that is S3-compatible and accessible to all workers in the cluster.

When a worker completes a task, it stores the outputs locally and uploads them to the shared store. If another worker later needs the same artifact, it retrieves it from the shared store instead of re-executing the task. Much like other tools on this class like CCache [16] Landseer's cached-artifacts are content-addressable, ensuring that equivalent identities across workers automatically reference the same stored artifact. Local caches implement a least-recently-used (LRU) eviction policy once disk space usage exceeds a configurable threshold. In contrast, the shared store maintains complete artifact lineage for reproducibility and restartability.

Challenge 5: Extensibility for Datasets, Models, Evaluators, and Tools Landseer is designed to support user-defined components without requiring modifications to the scheduler or execution engine. During the compilation of experiments, Landseer validates these specifications for schema correctness and stage compatibility. This plugin-style design allows for rapid expansion of benchmark coverage while ensuring reproducibility and compatibility.

6 Evaluation

In this Section, we study how Landseer can integrate TML defenses into a pipeline, based on clearly defined criteria for structural composability, to then discover qualitative composability insights. We evaluate Landseer's ability to detect interference by both validating and refuting prior interference findings, while also identifying new types of interference based on empirical data.

6.1 Experimental Setup

Datasets. Tools are tested on the CIFAR-10 dataset, a standard and widely used benchmark dataset for image classification tasks.

Table 2: Taxonomy of Composability Issues In Practice and Suggestions for Improvement

Composability class	Issues found in practice	Suggestions for improvement
Available	~30% lacked public artifacts; some only had third-party implementations [11, 21, 73, 81, 109, 111]	Standardize official artifact release
Reproducible	Software bugs, env. mismatch, missing configuration [52, 82, 110]	Provide validated code with clear instructions and requirements
Replicable	Lack of end-to-end integration guidance [13]	Provide minimal pipelines and integration examples
Self-contained	All tools were containerizable, though with varying effort [51]	In addition to source code, provide ready-to-use container images where feasible
Extensible	Some tools were limited by architecture-specific assumptions or requirements [1, 35]	Promote modular, model-agnostic designs where possible
Modular	Cross-stage dependencies reduced modularity (e.g., requiring specific preprocessing and training choices) [35, 105]	Enforce stage separation where possible and document dependencies

Model Architecture. We use a CIFAR-style Residual Network (ResNet), ResNet-20, consisting of an initial 3×3 convolution followed by three stages of residual blocks with 16, 32, and 64 channels. The model has a fully connected classification layer, with weights initialized using Kaiming initialization [43]. For differentially private training using Opacus [1], batch normalization layers are configured with *track_running_stats=False*. The modification avoids cross-batch information leakage through stored statistics and ensures that normalization depends only on the current mini-batch, which is more compatible with the per-sample gradient computations required by DP-SGD.

Hyperparameters. The baseline model is trained for 200 epochs with a batch size of 128 and an initial learning rate of 0.1. Standard CIFAR-10 data normalization was applied to both training and test sets. Data augmentation for training includes random cropping and random horizontal flipping. Hyperparameters for defenses tested in Landseer are listed in Table 6.

For the qualitative composability studies, we set the metric thresholds t_l and t_h (from Section 4.3) to 2% and 5%, respectively.

6.2 Structural Composability

We now empirically explore the landscape of TML tools and study their structural composability. Thus, we first build a collection of tools from both industry and academia that address all the defense axes in TML (as reviewed in Section 2). Then, we study our ability to integrate them into an MLOps pipeline to identify the patterns and techniques that allow for structural composability. As an added result, we also showcase the state of the tool ecosystem in terms of their composability.

6.2.1 TML Tool Selection & Integration Requirements. We begin our composability study by collecting TML tools from white and grey literature. This includes academic publications in various venues for both machine learning and computer security in order to identify existing solutions. Further, we enrich these solutions with bespoke

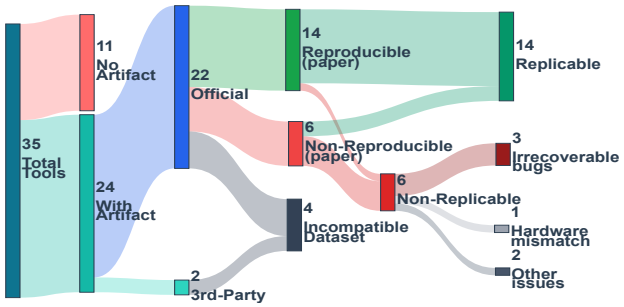

Figure 3: Summary of structural composability. Definitions for each category, as well as the flow and steps provided to assess composability are described in Section 6.2

systems from major open source machine learning frameworks (such as PyTorch). To ensure comprehensive and representative coverage, we survey work published over the past 12 years in top-tier conferences, journals, and other leading venues in machine learning, artificial intelligence, computer vision, and security, as summarized in Table 7. Specifically, we focus on identifying prior work addressing TML tools for evasion robustness, outlier robustness, privacy, watermarking, fingerprinting, group fairness, and explanation. We select TML tools to capture a representative sample that covers trustworthy tools across all ML axes and our structural categorization (as summarized in Table 5).

6.2.2 Integration Barriers During Onboarding. We define the structural composability criteria for tools and frameworks that implement each TML defense and motivate them through analysis of studied tools. Each tool should feature an open-source implementation that delivers reproducible results. Also, the tool should integrate into an ML pipeline and work alongside other techniques.

Specifically, we define the identify the following requirements for integration:

- *Available*: Publicly available source code. This may be an official artifact from the authors or a third-party artifact.
- *Reproducible*: Executable in a specific environment, and producing the reported results with the original code and data.
- *Replicable*: Executable in a specific environment with consistent results using different data and/or modified code.
- *Self-contained*: Functional within an isolated environment (e.g., a Docker container [64]).
- *Extensible*: Applicable to different datasets within a data modality (e.g., images) and model architectures.
- *Modular*: It can be incorporated into one or more of the ML pipeline stages with described inputs and outputs to be passed into and out of its environment.

We treat these properties as forming a partially ordered hierarchy rather than a strictly linear one. Some requirements are inherently hierarchical; for example, an artifact must be available to be reproducible or replicable. Others are imposed by Landseer 's criteria: an artifact that is not self-contained is not considered structurally composable, even if it is otherwise extensible. Additionally, reproducibility and replicability are treated as being at the same level, since an artifact may not reproduce the original results exactly but can still yield consistent results under variation and thus remain usable within a pipeline (Note two non-reproducible papers marked as replicable on the top part of Figure 3).

Defenses are categorized by their pipeline stage, but many are not *stage-self-contained* within that stage. This makes it challenging to use them modularly in Landseer. A common issue is cross-stage coupling, where a method that claims to operate at one stage may actually produce a partially trained model, thereby dictating a fixed fine-tuning approach. That would mean During-Training, specific optimizers, loss functions, and schedules must be employed. This practice, which spans Pre-Training and During-Training, undermines modular assumptions. For instance, adversarial contrastive Pre-Training [44] necessitates particular downstream training decisions linked to its design. A summary of additional barriers encountered includes:

- **Non-standard I/O contracts:** bespoke formats, altered label spaces, or custom wrappers incompatible with generic pipelines;
- **Environment constraints:** reliance on legacy GPU capability or obsolete libraries;
- **Reproducibility gaps:** missing code, seeds, or configuration details.

These barriers, more of which are listed in Table 2, underscore the need for explicit *structural composability* criteria in selecting defenses for integration.

For each defense with publicly available artifacts, we first attempted to reproduce the results reported in the original paper. To assess reproducibility, we define an error tolerance that accounts for expected variability in machine learning pipelines. In particular, we consider a defense reproducible if its performance falls within a small deviation threshold of the reported results. We set this threshold to 3%, which reflects typical run-to-run variation observed in practice while remaining strict enough to detect substantive discrepancies. Minor deviations of this magnitude can arise from nondeterminism in GPU operations, floating-point precision differences across hardware, subtle changes in library versions, or undocumented preprocessing steps.

Some defenses, especially older ones, do not execute properly in modern machine learning toolchains due to deprecated APIs or outdated dependencies. For those cases, we modified the implementations as little as possible to ensure compatibility, without altering their core logic or methodology. To verify that these modifications did not affect performance, we re-ran the original experiments and confirmed that the results deviated from the published values by no more than $\tilde{3}$%.

Further, many defenses are implemented with author-chosen optimizers and learning-rate schedulers (e.g., SGD, Adam, AdamW, DP-SGD). This variation can complicate comparisons because differences in utility and robustness may arise from the optimizer or schedule rather than the defense itself. In this iteration of Landseer, we maintain the original optimizer and schedule from each paper to ensure accuracy. Currently, if two tools have different optimizers and this affects utility or defense, we will consider them as negatively interfering and hence not composable. However, we also acknowledge that systematically varying these factors is crucial for

exploring the interplay between optimizers and defenses, and their composability.

Lastly, given that some solutions are intended more as technology demonstrations rather than components to be directly integrated into a pipeline, we make minimal modifications to their implementations to allow for chaining of steps. For example, in [113], the defense code outputs an array of 0s and 1s to indicate whether a data point is an outlier (1) or not (0), but this output cannot be directly passed to the next step in the pipeline. To address this, we instruct Landseer to add a post-processing stage to remove outliers before passing the resulting artifact forward.

6.2.3 Structural Landscape. We present the results of the onboarding process in Figure 1. From a total of 35 original papers, 11 were excluded due to the absence of public artifacts. Of the remaining 24 papers, 22 had official artifacts and 2 third-party implementations. All 2 third-party implementations, along with 2 more official implementations, were dropped as they focused on tabular datasets, while Landseer focused on image datasets. Of the remaining 20 official artifacts, 14 were reproducible and replicable, and the remaining 6 were dropped due to irrecoverable bugs that made the artifacts fail to run. Of 14 replicable papers, some were included directly into Landseer without changes while the rest required minimal modifications, such as updating deprecated packages, updating python versions, and updating API function calls.

6.3 Qualitative Composability

We follow our structural composability evaluation with conducting qualitative evaluations to demonstrate Landseer capabilities. Specifically, we focus on two case studies. In the first case study, we compare Landseer results to existing work on pairwise defense combinations to evaluate if Landseer can reproduce prior conjectures and examine any discrepancies. In the second case study, we explore new combinations that are identified by Landseer with interesting properties that have not been examined in prior work, demonstrating the extensibility of Landseer.

6.3.1 Case Study 1: Comparison to Existing Pairwise Defense Combinations. We begin by evaluating Landseer's interference results against existing work on interactions between defenses and summarize our results in Table 3. In this table, we define and mark conflicting combinations as one in which one or more performance metrics degrade beyond the thresholds defined in Section 6.1, and not conflicting (aligning) as one with improved or unchanged metrics. To demonstrate the ability of Landseer to reproduce existing results, we perform a case study with five papers in literature by Duddu et al. [29], Szyller et al. [92], Lukas et al. [56], Hayes et al. [40], and Strobel et al. [88] that examine interactions of pairwise defenses. In particular, Duddu et al. [29] propose a non-experimental prediction technique, DEF\CON, that uses a tool questioning framework to explore interactions between pairwise defenses to determine if a combination of defenses will be aligning, and Szyller et al. [92] empirically examined the pairwise combination of model/data ownership verification with differentially private training and model evasion robustness, to find aligning or conflicting interactions. For each combination, we inspect when Landseer confirms or contradicts the results in prior work. As shown in Table 3, Landseer

Table 3: Comparison of Landseer findings on pairwise tool combinations with existing work. We indicate Landseer confirming (contradicting) prior work with ● (●).

Tool Category	Conflicting	Existing Work	
		Confirm	Contradict
AR_{in} / OR_{post}	Yes	● [29]	○
AR_{in} / FP_{dep}	Yes	● [56]	● [29, 92]
WM_{pre} / AR_{in}	Yes	● [29, 92]	○
AR_{in} / EX_{dep}	Yes	○	● [29]
OR_{post} / FP_{dep}	No	● [29]	○
WM_{pre} / OR_{post}	Yes	● [29]	○
OR_{post} / EX_{dep}	No	● [29]	○
FP_{dep} / EX_{dep}	No	● [29]	○
WM_{pre} / EX_{dep}	No	● [29]	○
OR_{pre} / DP_{in}	No	● [88]	○
AR_{in} / DP_{dep}	Yes	● [40, 88]	○
DP_{in} / AR_{post}	Yes	● [40]	○
DP_{in} / FP_{dep}	Yes	○	● [29, 92]
DP_{in} / EX_{dep}	No	● [29]	○
DP_{in} / OR_{post}	No	● [88]	○

obtains the same results for 13 different pairwise defense combinations, confirming the existence (lack) of interference in 8 (7) cases.

Confirmations. Duddu et al. [29] determine that model Explanation techniques applied after model training EX_{dep} do not conflict with watermarking WM_{pre}, differential privacy DP_{in}, poisoning robustness OR_{post}, or fingerprinting FP_{dep}. On the other hand, they report interference between watermarking applied before training WM_{pre} and robustness defenses (adversarial robustness AR_{in} and poisoning robustness OR_{post}). Similarly, their results show interference between adversarial robustness AR_{in} and backdoor removal OR_{post}. Landseer confirms these results, sharing four of the six tools they use in their evaluation, and differing in two. The two that differ are OR_{post} where they use a technique by Zheng et al. [115], and EX_{dep} where they use DeepLift [85]. Interestingly, Duddu et al. initially predict that the AR_{in}-OR_{post} combination does not result in interference based on their proposed model. However, their empirical results confirm the existence of interference, demonstrating the challenges of prediction models for defense interactions.

Szyller et al. [92] confirm the interference between watermarking WM_{pre} and adversarial robustness AR_{in}, since adversarial training suppresses introduced watermarks in most cases. Hayes et al. [40] and Strobel et al. [88] determine that adversarial robustness AR_{in}

and differential privacy DP_{dep} are interfering. Strobel et al. claim that adversarial training, a form of adversarial robustness technique, modifies model decision boundaries in a way that an attacker can exploit to craft successful inference attacks, and Hayes et al. claim that adversarial perturbations and clipping norms lead to poor model generalization.

Contradictions. Duddu et al. and Szyller et al. report no interference between adversarial robustness AR_{in} and fingerprinting FP_{dep}, whereas Landseer empirical data (using the same techniques: AR_{in} [110] and FP_{dep} [60]) contradicts this finding. We note that Lukas et al. [56], which also investigates this combination with different AR_{in} and FP_{dep} techniques on CIFAR10 and Resnet20, claim a conflicting interaction between the two tools. Duddu et al. report their result based on their prediction model and without performing any empirical evaluation for this setting. Szyller et al. [92] conduct experiments using a different adversarial robustness technique [59] but only report metrics for fingerprinting and do not include metrics for the adversarial robustness tool to confirm their claim. While the absence of this data does not necessarily invalidate their claim, the use of a different adversarial robustness technique could explain the discrepancy in results. Interference effects may depend on the specific TML techniques used, an aspect that future extensions of Landseer aim to investigate further.

In a new pairwise combination between differential privacy DP_{in} and fingerprinting FP_{dep}, both Duddu et al. and Szyller et al. claim an aligning combination, where Duddu et al. rely solely on their prediction model while Szyller et al. use empirical data to show fingerprinting is not affected, but not the value for differential privacy. Additionally, discrepancies with Landseer may stem from differences in experimental settings, including datasets, model architectures, and evaluation metrics. While overlapping components (e.g., CIFAR-10 and ResNet-20) are used, the exact model–dataset mappings in Szyller et al. are unclear. Moreover, they evaluate differential privacy using classification accuracy, whereas Landseer uses membership inference attack AUC, making comparison difficult.

In the case of adversarial robustness and explanation techniques, there is a contradiction between Landseer's results and [29]. Duddu et al. [29] claim that there is no interference between adversarial robustness AR_{in} and explanation EX_{dep}, whereas Landseer discovered interference mainly on the explanation defense. The difference in interference claims could be attributed to the use of different explanation techniques and experimental setups. [29] used the DeepLift [85] technique and tested on FMNIST and UKTFACE with a 2-layer CNN model, whereas Landseer implemented DeepSHAP [58] on CIFAR10 and a Resnet20 model.

6.3.2 Case Study 2: New Defense Combinations. In our next case study, we examine new defense combinations identified by Landseer that, to the best of our knowledge, have not been explored in prior work. We show that overlooked factors such as the stage at which a defense is applied or the order of defenses in the same stage can result in different interference patterns.

Unexplored Pairwise Combinations. In Table 4 we show examples of pairwise defense combination that have not been explored previously but prove to be insightful. Specifically, Landseer discovers that incorporating outlier removal in the Pre-Training stage

Table 4: Subset of new defense combinations with interference state. Interference will be indicated as follows: Pairwise (PW), Ordering (OI), and Global (GI). For each metric, ↑ (⇓) indicates improved (degraded), + + (- -) represents severe improvement (degradation), + (-) refers to moderate improvement (degradation), and ≡ denotes no change. A bidirectional arrow ⇔ indicates that swapping the order of tools in the same defense stage will observe the metric changes recorded.

Combination	Interf.	Metric Change
OR_{pre} / AR_{in}	PW	$\mathbf{m^{ar}}$ ↑++, $\mathbf{m^{ev}}$ ↑++
AR_{post} / FP_{dep}	PW	$\mathbf{m^{ev}}$ ≡, $\mathbf{m^{fp}}$ ≡
OR_{pre}+WM_{pre}	PW	$\mathbf{m^{ar}}$ ≡, $\mathbf{m^{wm}}$ ≡
AR_{in} / AR_{post}	PW	$\mathbf{m^{ev}}$ ↑++
AR_{post} / DP_{dep}	PW	$\mathbf{m^{ev}}$ ↓- -,$\mathbf{m^{dp}_{mia}}$ ≡
OR_{pre} / EX_{dep}	PW	$\mathbf{m^{ar}}$ ≡, $\mathbf{m^{ex}}$ ↓- -
WM_{pre}+OR_{pre} / AR_{in} / AR_{post}+OR_{post}	GI	$\mathbf{m^{wm}}$ ≡, $\mathbf{m^{ar}}$ ≡, $\mathbf{m^{ev}}$ ↓- - , $\mathbf{m^{ou}}$ ≡
WM_{pre}+OR_{pre} / AR_{in} / AR_{post}+OR_{post} / FP_{dep}	GI	$\mathbf{m^{wm}}$ ≡, $\mathbf{m^{ar}}$ ≡, $\mathbf{m^{ev}}$ ↑++, $\mathbf{m^{ou}}$ ≡
WM_{pre}+OR_{pre} / AR_{in} / AR_{post}+OR_{post} / EX_{dep}	GI	$\mathbf{m^{wm}}$ ≡, $\mathbf{m^{ar}}$ ≡, $\mathbf{m^{ev}}$ ↑++, $\mathbf{m^{ou}}$ ≡
OR_{pre} ⇔ WM_{pre} / OR_{post}	OI	$\mathbf{m^{ar}}$ ↓- , $\mathbf{m^{wm}}$ ↑++
OR_{pre} ⇔ WM_{pre} / AR_{in}	OI	$\mathbf{m^{ar}}$ ≡ , $\mathbf{m^{wm}}$ ↑++
WM_{pre} ⇔ OR_{pre} / AR_{post}+OR_{post}	OI	$\mathbf{m^{ar}}$ ↑+ , $\mathbf{m^{wm}}$ ↓- -

OR_{pre} and adversarial training AR_{in} significantly improve the corresponding metrics. This result is noteworthy since prior conclusions by Duddu et al. (also confirmed by Landseer) indicate that applying outlier robustness techniques Post-Training OR_{post} results in a conflicting combination with AR_{in} (Table 3). The same trend is seen where a previously explored combination of adversarial training AR_{in} and fingerprinting FP_{dep} may give conflicting results (Table 3), but applying the adversarial robustness technique in the post stage AR_{post} could be a better option for aligning the defenses as our new results indicate in Table 4.

Landseer reveals another interesting pairwise combination where applying watermaking WM_{pre} and outlier removal OR_{pre} (both in the Pre-Training stage) does not degrade either metric, while previously explored combinations of watermaking WM_{pre} and outlier robustness OR_{post} observed a conflicting combination (Table 3).

For defenses with the same objective, we further notice that their pairwise combination could be beneficial as a layered defense strategy. For example, Table 4 shows that applying adversarial robustness techniques to During-Training AR_{in} and Post-Training AR_{post} results in a significant improvement in the adversarial robustness metric compared to combinations where only one is present.

Other previously unexplored pairwise combinations in Table 4, namely AR_{post}/DP_{dep} and OR_{pre}/EX_{dep}), indicate interference with degraded metric values. Comparing this insight with prior results in Table 3 with the same defense in other stages reveals agreement in one case (conflicting AR_{in}/DP_{dep}) and disagreement in the other case (aligning OR_{post}/EX_{dep}), hinting at other factors that may be at play here worth exploring in future work.

Combinations with Global Interference Landseer is also able to explore combinations of 2+ tools which have been mostly overlooked in prior work. Our results reveal clear stage-dependent patterns. For example, Pre-Training tools (WM_{pre}, OR_{pre}) and During-Training (DP_{in},AR_{in}) tools consistently show interference when combined with other tools (>95% of combinations). This result indicates that defenses applied at later stages may have better composability with other techniques which can be an interesting avenue for future work. We include a few examples of combinations with 5+ tools in Table 4 that show interesting global interference trends (marked with GI). The three focus tools here, namely EX_{dep}, FP_{dep}, and AR_{post}, do not exhibit global interference except in the specific arrangement shown in the table. Notably, the addition of FP_{dep} and EX_{dep} improve the robustness metric. More detailed exploration of these combinations could reveal more insights about the interactions of defenses when several coexist in the pipeline.

Combinations with Ordering Interference Finally, we present our results for ordering interference (marked by OI in Table 4). Notably, applying OR_{pre} and WM_{pre} alone does not result in ordering interference. However, in the presence of OR_{post} or AR_{dur}, the ordering between OR_{pre} and WM_{pre} becomes important, with OR_{pre} before WM_{pre} producing more favorable outcomes.

7 Discussion

Landseer sits at the crossroads of many efforts to improve TML defenses in the MLOps context. Naturally, this suggests there exist various efforts and approaches to improve the status quo. Particularly, work on replicability and composability has been broached in various ways, and exploring their overlap — beyond what a typical related work section would — is paramount. Lastly, we also explore ways in which Landseer can be improved or modified.

Contemporary Work on Reproducibility. Existing work has explored the way in which TML defenses papers are reproducible in practice. Notably, Olszewski et al. [70] carried out a large-scale replication experiment similar to our structural reproducibility work. Our results complement some of the findings in that work, in that we found similar patterns and challenges during our replication phase — they found that about 20% of papers were replicable, while we found that about 40% were so. However, Landseer expands this notion by also providing visibility on how these papers can integrate into an MLOps pipeline.

Contemporary Work on Composability. Similarly, there has been an uptick in work relating to our notions of TML defenses' composability. In this regard, contemporary work (like [29, 66, 90]) mostly provides a manual analysis approach to explore piecemeal combinations of tools. Instead, Landseer provides a systematic framework that: 1) Allows researchers to quickly test the composability of novel systems by packaging their TML defenses as a module. 2) Allows practitioners to test a specific pipeline configuration to find sources of interference 3) Provides guidance on defense alternatives of pipeline configurations that minimize interference

Tuning TML defenses Parameters to Improve Composability. An intuitive notion to improve composability may be by tweaking TML defenses individual parameters. For example, by reducing the σ for a differential privacy defense, we may reduce its interference on other TML defenses. However, searching for optimal operations and determining acceptable thresholds would require substantial changes to the way that solutions are composed today, and how the Landseer pipeline executor explores what effectively is an n-dimensional hyperspace. We defer these extensions to Landseer to future work as a consequence.

8 Related Work

Much existing literature focuses on single defenses for the known threats in the ML/AI supply chain such as adversarial robustness for adversarial attacks [22, 83, 103, 114], outlier removal and pruning for poisoning and backdoor attacks [42, 50, 101, 104], among others [2, 52, 55, 74, 80, 89] as discussed in section 3. Multiple defenses are needed to properly secure ML models and AI systems.

Recent work focused on combining at most two defenses and studying the interactions/conflicts between those defenses [19, 33, 68]. Szyller et al. [92] study pairwise combination of ownership verification with differential privacy and evasion robustness, finding frequent conflicts. Duddu et al. [28] show that defenses targeting one risk (security, privacy, fairness) can unintentionally affect others. In [29], Duddu et al. propose a non-experimental framework to assess defense compatibility. It is limited to two defenses and has little validation for larger combinations. Cuong Tran et al. [96] study trade-offs between evasion robustness and fairness, proposing a framework for systems with good trade-offs between robustness and fairness. Additionally, work by Khan et al. [39] highlights the complexity of multi-objective defense design, showing that improving one objective (e.g., privacy via regularization) can degrade utility or fairness in image classification, reinforcing the need for careful composition of defenses.

Despite significant efforts in the related work to address the challenge of combining ML defenses, there is still no infrastructure that enables the seamless composition of multiple ML defenses with empirical guarantees. Existing combination techniques are limited to two defenses at a time, and there is no systematic way to evaluate the effects of adding more defenses or tuning their parameters within a unified framework.

9 Conclusion & Future Work

This paper introduces Landseer, a framework to systematically explore TML tool composability. Landseer can easily integrate a defense in reference machine learning pipelines, and compute an exhaustive combination of all configurations with all other tools. Afterwards, it identifies their interference and catalogs it in a pipeline-oriented taxonomy. Using Landseer we were able to identify various sources of TML defenses interferences, as well as contrast with contemporary work.

Though the current implementation of Landseer is relatively coarse, for it does not allow exploring low-level semantics that may influence interference, such as optimizers, batching approach, TML defenses security parameters, etc., we envision future extensions of the framework will allow us to provide insight into these. Similarly, the inclusion of a larger corpus of work may also allow us to improve the insights we derive from our interference function. However, as is, Landseer is already able to provide useful insights for researchers and practitioners around ML composability.

References

[1] Martín Abadi, Andy Chu, Ian J. Goodfellow, H. B. McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep Learning with Differential Privacy. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security* (2016). https://api.semanticscholar.org/CorpusID:207241585

[2] Yossi Adi, Carsten Baum, Moustapha Cissé, Benny Pinkas, and Joseph Keshet. 2018. Turning Your Weakness Into a Strength: Watermarking Deep Neural Networks by Backdooring. In *USENIX Security Symposium*. https://api.semanticscholar.org/CorpusID:3322503

[3] Alekh Agarwal, Alina Beygelzimer, Miroslav Dudík, John Langford, and Hanna Wallach. 2018. A Reductions Approach to Fair Classification. arXiv:1803.02453 [cs.LG] https://arxiv.org/abs/1803.02453

[4] David Alvarez-Melis and T. Jaakkola. 2018. Towards Robust Interpretability with Self-Explaining Neural Networks. *ArXiv* abs/1806.07538 (2018). https://api.semanticscholar.org/CorpusID:49324194

[5] Eslam Amer and Ivan Zelinka. 2020. A dynamic Windows malware detection and prediction method based on contextual understanding of API call sequence. *Computers & Security* 92 (2020), 101760. doi:10.1016/j.cose.2020.101760

[6] Julia Angwin, Jeff Larson, Surya Mattu, and Lauren Kirchner. 2022. Machine bias. In *Ethics of data and analytics*. Auerbach Publications, 254–264.

[7] Apptainer. 2023. Apptainer. https://apptainer.org/

[8] Giovanni Apruzzese, Pavel Laskov, Edgardo Montes de Oca, Wissam Mallouli, Luis Brdalo Rapa, Athanasios Vasileios Grammatopoulos, and Fabio Di Franco. 2023. The Role of Machine Learning in Cybersecurity. *Digital Threats* 4, 1, Article 8 (March 2023), 38 pages. doi:10.1145/3545574

[9] Solon Barocas, Moritz Hardt, and Arvind Narayanan. 2023. *Fairness and machine learning: Limitations and opportunities*. MIT press.

[10] Battista Biggio, Blaine Nelson, and Pavel Laskov. 2012. Poisoning attacks against support vector machines. *arXiv preprint arXiv:1206.6389* (2012).

[11] Eitan Borgnia, Valeriia Cherepanova, Liam H. Fowl, Amin Ghiasi, Jonas Geiping, Micah Goldblum, Tom Goldstein, and Arjun Gupta. 2020. Strong Data Augmentation Sanitizes Poisoning and Backdoor Attacks Without an Accuracy Tradeoff. *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (2020), 3855–3859. https://api.semanticscholar.org/CorpusID:227054251

[12] Barbara Bravi. 2024. Development and use of machine learning algorithms in vaccine target selection. *npj Vaccines* 9, 1 (2024), 15.

[13] Clément L. Canonne, Gautam Kamath, and Thomas Steinke. 2020. The Discrete Gaussian for Differential Privacy. *ArXiv* abs/2004.00010 (2020). https://api.semanticscholar.org/CorpusID:214743526

[14] Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. 2019. IPGuard: Protecting the Intellectual Property of Deep Neural Networks via Fingerprinting the Classification Boundary. *ArXiv* abs/1910.12903 (2019). https://api.semanticscholar.org/CorpusID:204960658

[15] Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. 2019. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *28th USENIX security symposium (USENIX security 19)*. 267–284.

[16] CCache Developers. 2026. ccache — compiler cache_2026. https://ccache.dev/

[17] Huili Chen, Bita Darvish Rouhani, and Farinaz Koushanfar. 2018. BlackMarks: Blackbox Multibit Watermarking for Deep Neural Networks. *ArXiv* abs/1904.00344 (2018). https://api.semanticscholar.org/CorpusID:90260955

[18] Huili Chen, Bita Darvish Rouhani, and Farinaz Koushanfar. 2018. DeepMarks: A Digital Fingerprinting Framework for Deep Neural Networks. *IACR Cryptol. ePrint Arch.* 2018 (2018), 322. https://api.semanticscholar.org/CorpusID:4759464

[19] Huiqiang Chen, Tianqing Zhu, Tao Zhang, Wanlei Zhou, and Philip S. Yu. 2023. Privacy and Fairness in Federated Learning: On the Perspective of Tradeoff. *ACM Comput. Surv.* 56, 2, Article 39 (Sept. 2023), 37 pages. doi:10.1145/3606017

[20] Christopher A Choquette-Choo, Florian Tramer, Nicholas Carlini, and Nicolas Papernot. 2021. Label-only membership inference attacks. In *International conference on machine learning*. PMLR, 1964–1974.

[21] Moustapha Cissé, Piotr Bojanowski, Edouard Grave, Yann Dauphin, and Nicolas Usunier. 2017. Parseval Networks: Improving Robustness to Adversarial Examples. *ArXiv* abs/1704.08847 (2017). https://api.semanticscholar.org/CorpusID:26714567

[22] Gilad Cohen and Raja Giryes. 2021. Simple Post-Training Robustness using Test Time Augmentations and Random Forest. *2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)* (2021), 3984–3994. https://api.semanticscholar.org/CorpusID:244709418

[23] Gilad Cohen and Raja Giryes. 2024. Simple post-training robustness using test time augmentations and random forest. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 3996–4006.

[24] Jeremy M. Cohen, Elan Rosenfeld, and J. Zico Kolter. 2019. Certified Adversarial Robustness via Randomized Smoothing. *ArXiv* abs/1902.02918 (2019). https://api.semanticscholar.org/CorpusID:59842968

[25] Jesse Davis and Mark Goadrich. 2006. The relationship between Precision-Recall and ROC curves. In *Proceedings of the 23rd international conference on Machine learning*. 233–240.

[26] Kang Liu Brendan Dolan-Gavitt and Siddharth Garg. 2018. Fine-Pruning: Defending Against Backdooring Attacks on Deep. In *Research in Attacks, Intrusions, and Defenses: 21st International Symposium, RAID 2018, Heraklion, Crete, Greece, September 10-12, 2018, Proceedings*, Vol. 11050. Springer, 273.

[27] Flávio du Pin Calmon, Dennis Wei, Bhanukiran Vinzamuri, Karthikeyan Natesan Ramamurthy, and Kush R. Varshney. 2017. Optimized Pre-Processing for Discrimination Prevention. In *Neural Information Processing Systems*. https://api.semanticscholar.org/CorpusID:3801798

[28] Vasisht Duddu, Sebastian Szyller, and N. Asokan. 2024. SoK: Unintended Interactions among Machine Learning Defenses and Risks. In *2024 IEEE Symposium on Security and Privacy (SP)*. 2996–3014. doi:10.1109/SP54263.2024.00243

[29] Vasisht Duddu, Rui Zhang, and N Asokan. 2024. Combining Machine Learning Defenses without Conflicts. *arXiv preprint arXiv:2411.09776* (2024).

[30] Kevin Eykholt, Ivan Evtimov, Earlence Fernandes, Bo Li, Amir Rahmati, Chaowei Xiao, Atul Prakash, Tadayoshi Kohno, and Dawn Song. 2018. Robust Physical-World Attacks on Deep Learning Models. arXiv:1707.08945 [cs.CR] https://arxiv.org/abs/1707.08945

[31] Samuel G. Finlayson, John D. Bowers, Joichi Ito, Jonathan L. Zittrain, Andrew L. Beam, and Isaac S. Kohane. 2019. Adversarial attacks on medical machine learning. *Science* 363, 6433 (2019), 1287–1289. arXiv:https://www.science.org/doi/pdf/10.1126/science.aaw4399 doi:10.1126/science.aaw4399

[32] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. 2015. Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures. *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security* (2015). https://api.semanticscholar.org/CorpusID:207229839

[33] Alex Gittens, Bülent Yener, and Moti Yung. 2022. An Adversarial Perspective on Accuracy, Robustness, Fairness, and Privacy: Multilateral-Tradeoffs in Trustworthy ML. *IEEE Access* 10 (2022), 120850–120865. doi:10.1109/ACCESS.2022.3218715

[34] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572* (2014).

[35] Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. 2017. Badnets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733* (2017).

[36] Chuan Guo, Mayank Rana, Moustapha Cissé, and Laurens van der Maaten. 2018. Countering Adversarial Images using Input Transformations. *ArXiv* abs/1711.00117 (2018). https://api.semanticscholar.org/CorpusID:12308095

[37] Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor Wai-Hung Tsang, and Masashi Sugiyama. 2018. Co-teaching: Robust training of deep neural networks with extremely noisy labels. In *Neural Information Processing Systems*. https://api.semanticscholar.org/CorpusID:52065462

[38] Moritz Hardt, Eric Price, and Nathan Srebro. 2016. Equality of Opportunity in Supervised Learning. *ArXiv* abs/1610.02413 (2016). https://api.semanticscholar.org/CorpusID:7567061

[39] Ahmad Hassanpour, Amir Zarei, Khawla Mallat, Anderson Santana de Oliveira, and Bian Yang. 2024. The Impact of Generalization Techniques on the Interplay Among Privacy, Utility, and Fairness in Image Classification. *arXiv preprint arXiv:2412.11951* (2024).

[40] Jamie Hayes, Borja Balle, and M. Pawan Kumar. 2022. Learning to be adversarially robust and differentially private. *CoRR* abs/2201.02265 (2022). arXiv:2201.02265 https://arxiv.org/abs/2201.02265

[41] Naoise Holohan, Stefano Braghin, Pól Mac Aonghusa, and Killian Levacher. 2019. Diffprivlib: the IBM differential privacy library. *arXiv preprint arXiv:1907.02444* (2019).

[42] Sanghyun Hong, Varun Chandrasekaran, Yigitcan Kaya, Tudor Dumitras, and Nicolas Papernot. 2020. On the Effectiveness of Mitigating Data Poisoning Attacks with Gradient Shaping. *ArXiv* abs/2002.11497 (2020). https://api.semanticscholar.org/CorpusID:211506328

[43] Yerlan Idelbayev. [n. d.]. Proper ResNet Implementation for CIFAR10/CIFAR100 in PyTorch. https://github.com/akamaster/pytorch_resnet_cifar10. Accessed: 2025-05-06.

[44] Ziyu Jiang, Tianlong Chen, Ting Chen, and Zhangyang Wang. 2020. Robust pre-training by adversarial contrastive learning. *Advances in neural information processing systems* 33 (2020), 16199–16210.

[45] Hamid Khayyam, Bahman Javadi, Mahdi Jalili, and Reza N. Jazar. 2020. *Artificial Intelligence and Internet of Things for Autonomous Vehicles*. Springer International Publishing, Cham, 39–68.

[46] Been Kim, Martin Wattenberg, Justin Gilmer, Carrie J. Cai, James Wexler, Fernanda B. Viégas, and Rory Sayres. 2017. Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV). In *International Conference on Machine Learning*. https://api.semanticscholar.org/CorpusID:51737170

[47] Jonathan Knauer, Phillip Rieger, Hossein Fereidooni, and Ahmad-Reza Sadeghi. 2024. Phantom: Untargeted Poisoning Attacks on Semi-Supervised Learning. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 615–629.

[48] Chieh-Hsin Lai, Dongmian Zou, and Gilad Lerman. 2019. Robust Subspace Recovery Layer for Unsupervised Anomaly Detection. *ArXiv* abs/1904.00152 (2019). https://api.semanticscholar.org/CorpusID:90262267

[49] Kornel Lewicki, Michelle Seng Ah Lee, Jennifer Cobbe, and Jatinder Singh. 2023. Out of Context: Investigating the Bias and Fairness Concerns of "Artificial Intelligence as a Service". In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 135, 17 pages. doi:10.1145/3544548.3581463

[50] Yige Li, Nodens Koren, L. Lyu, Xixiang Lyu, Bo Li, and Xingjun Ma. 2021. Neural Attention Distillation: Erasing Backdoor Triggers from Deep Neural Networks. *ArXiv* abs/2101.05930 (2021). https://api.semanticscholar.org/CorpusID:231627799

[51] Yige Li, Xixiang Lyu, Xingjun Ma, Nodens Koren, L. Lyu, Bo Li, and Yugang Jiang. 2023. Reconstructive Neuron Pruning for Backdoor Defense. In *International Conference on Machine Learning*. https://api.semanticscholar.org/CorpusID:258865980

[52] Zi-Han Lin, Sivakanth Gopi, Janardhan Kulkarni, Harsha Nori, and Sergey Yekhanin. 2023. Differentially Private Synthetic Data via Foundation Model APIs 1: Images. *ArXiv* abs/2305.15560 (2023). https://api.semanticscholar.org/CorpusID:258888127

[53] Zachary C Lipton. 2018. The mythos of model interpretability: In machine learning, the concept of interpretability is both important and slippery. *Queue* 16, 3 (2018), 31–57.

[54] Lan Liu, Yi Wang, Gaoyang Liu, Kai Peng, and Chen Wang. 2022. Membership inference attacks against machine learning models via prediction sensitivity. *IEEE Transactions on Dependable and Secure Computing* 20, 3 (2022), 2341–2347.

[55] Nils Lukas, Edward Jiang, Xinda Li, and Florian Kerschbaum. 2021. SoK: How Robust is Image Classification Deep Neural Network Watermarking? *2022 IEEE Symposium on Security and Privacy (SP)* (2021), 787–804. https://api.semanticscholar.org/CorpusID:236975869

[56] Nils Lukas, Yuxuan Zhang, and Florian Kerschbaum. 2019. Deep Neural Network Fingerprinting by Conferrable Adversarial Examples. *ArXiv* abs/1912.00888 (2019). https://api.semanticscholar.org/CorpusID:208527270

[57] Scott M Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. *Advances in neural information processing systems* 30 (2017).

[58] Scott M Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf

[59] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2017. Towards Deep Learning Models Resistant to Adversarial Attacks. *ArXiv* abs/1706.06083 (2017). https://api.semanticscholar.org/CorpusID:3488815

[60] Pratyush Maini. 2021. Dataset Inference: Ownership Resolution in Machine Learning. *ArXiv* abs/2104.10706 (2021). https://api.semanticscholar.org/CorpusID:231609191

[61] Shagufta Mehnaz, Ninghui Li, and Elisa Bertino. 2020. Black-box model inversion attribute inference attacks on classification models. *arXiv preprint arXiv:2012.03404* (2020).

[62] Luca Melis, Congzheng Song, Emiliano De Cristofaro, and Vitaly Shmatikov. 2018. Exploiting Unintended Feature Leakage in Collaborative Learning. *2019 IEEE Symposium on Security and Privacy (SP)* (2018), 691–706. https://api.semanticscholar.org/CorpusID:53099247

[63] Dongyu Meng and Hao Chen. 2017. MagNet: A Two-Pronged Defense against Adversarial Examples. *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (2017). https://api.semanticscholar.org/CorpusID:3583538

[64] Dirk Merkel. 2014. Docker: lightweight Linux containers for consistent development and deployment. *Linux J.* 2014, 239, Article 2 (March 2014).

[65] Ilya Mironov. 2017. Rényi differential privacy. In *2017 IEEE 30th computer security foundations symposium (CSF)*. IEEE, 263–275.

[66] Manish Nagireddy, Moninder Singh, Samuel C. Hoffman, Evaline Ju, Karthikeyan Natesan Ramamurthy, and Kush R. Varshney. 2023. Function Composition in Trustworthy Machine Learning: Implementation Choices, Insights, and Questions. arXiv:2302.09190 [cs.LG] https://arxiv.org/abs/2302.09190

[67] Akm Iqtidar Newaz, Nur Imtiazul Haque, Amit Kumar Sikder, Mohammad Ashiqur Rahman, and A. Selcuk Uluagac. 2020. Adversarial Attacks to Machine Learning-Based Smart Healthcare Systems. In *GLOBECOM 2020 - 2020 IEEE Global Communications Conference*. 1–6. doi:10.1109/GLOBECOM42002.2020.9322472

[68] Maximilian Noppel and Christian Wressnegger. 2024. SoK: Explainable Machine Learning in Adversarial Environments. In *2024 IEEE Symposium on Security and Privacy (SP)*. 2441–2459. doi:10.1109/SP54263.2024.00021

[69] Daryna Oliynyk, Rudolf Mayer, and Andreas Rauber. 2023. I Know What You Trained Last Summer: A Survey on Stealing Machine Learning Models and Defences. *ACM Comput. Surv.* 55, 14s, Article 324 (July 2023), 41 pages. doi:10.1145/3595292

[70] Daniel Olszewski, Allison Lu, Carson Stillman, Kevin Warren, Cole Kitroser, Alejandro Pascual, Divyajyoti Ukirde, Kevin Butler, and Patrick Traynor. 2023. "Get in Researchers; We're Measuring Reproducibility": A Reproducibility Study of Machine Learning Papers in Tier 1 Security Conferences *(CCS '23)*. Association for Computing Machinery, New York, NY, USA, 3433–3459. doi:10.1145/3576915.3623130

[71] Open Container Initiative. 2015. Open Container Initiative. https://opencontainers.org.

[72] Nicolas Papernot, Patrick McDaniel, Arunesh Sinha, and Michael P. Wellman. 2018. SoK: Security and Privacy in Machine Learning. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*. 399–414. doi:10.1109/EuroSP.2018.00035

[73] Nicolas Papernot, Patrick Mcdaniel, Xi Wu, Somesh Jha, and Ananthram Swami. 2015. Distillation as a Defense to Adversarial Perturbations Against Deep Neural Networks. *2016 IEEE Symposium on Security and Privacy (SP)* (2015), 582–597. https://api.semanticscholar.org/CorpusID:2672720

[74] Zirui Peng, Shaofeng Li, Guoxing Chen, Cheng Zhang, Haojin Zhu, and Minhui Xue. 2022. Fingerprinting Deep Neural Networks Globally via Universal Adversarial Perturbations. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2022), 13420–13429. https://api.semanticscholar.org/CorpusID:246904661

[75] Dana Pessach and Erez Shmueli. 2022. A review on fairness in machine learning. *ACM Computing Surveys (CSUR)* 55, 3 (2022), 1–44.

[76] Vitali Petsiuk, Abir Das, and Kate Saenko. 2018. Rise: Randomized input sampling for explanation of black-box models. *arXiv preprint arXiv:1806.07421* (2018).

[77] Amir Masoud Rahmani, Efat Yousefpoor, Mohammad Sadegh Yousefpoor, Zahid Mehmood, Amir Haider, Mehdi Hosseinzadeh, and Rizwan Ali Naqvi. 2021. Machine Learning (ML) in Medicine: Review, Applications, and Challenges. *Mathematics* 9, 22 (2021). doi:10.3390/math9222970

[78] Ali Rezaei Nasab, Maedeh Dashti, Mojtaba Shahin, Mansooreh Zahedi, Hourieh Khalajzadeh, Chetan Arora, and Peng Liang. 2025. Fairness Concerns in App Reviews: A Study on AI-Based Mobile Apps. *ACM Trans. Softw. Eng. Methodol.* 34, 2, Article 51 (Jan. 2025), 30 pages. doi:10.1145/3690633

[79] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. " Why should i trust you?" Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 1135–1144.

[80] Andrew Slavin Ross, Michael C. Hughes, and Finale Doshi-Velez. 2017. Right for the Right Reasons: Training Differentiable Models by Constraining their Explanations. *ArXiv* abs/1703.03717 (2017). https://api.semanticscholar.org/CorpusID:7053611

[81] Bita Darvish Rouhani, Huili Chen, and Farinaz Koushanfar. 2019. DeepSigns: An End-to-End Watermarking Framework for Ownership Protection of Deep Neural Networks. *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (2019). https://api.semanticscholar.org/CorpusID:102347976

[82] Alexandre Sablayrolles, Matthijs Douze, Cordelia Schmid, and Hervé Jégou. 2020. Radioactive data: tracing through training. In *International Conference on Machine Learning*. PMLR, 8326–8335.

[83] Ali Shafahi, Mahyar Najibi, Amin Ghiasi, Zheng Xu, John P. Dickerson, Christoph Studer, Larry S. Davis, Gavin Taylor, and Tom Goldstein. 2019. Adversarial Training for Free!. In *Neural Information Processing Systems*. https://api.semanticscholar.org/CorpusID:139102395

[84] R. Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2016. Membership Inference Attacks Against Machine Learning Models. *2017 IEEE Symposium on Security and Privacy (SP)* (2016), 3–18. https://api.semanticscholar.org/CorpusID:10488675

[85] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. 2017. Learning important features through propagating activation differences *(ICML'17)*. JMLR.org, 3145–3153.

[86] Justin Sirignano and Rama Cont. 2021. Universal features of price formation in financial markets: perspectives from deep learning. In *Machine learning and AI in finance*. Routledge, 5–15.

[87] Jacob Steinhardt, Pang Wei W Koh, and Percy S Liang. 2017. Certified defenses for data poisoning attacks. *Advances in neural information processing systems* 30 (2017).

[88] Martin Strobel and Reza Shokri. 2022. Data Privacy and Trustworthy Machine Learning. *IEEE Security & Privacy* 20, 5 (2022), 44–49. doi:10.1109/MSEC.2022.3178187

[89] Rishabh Subramanian. 2023. Have the cake and eat it too: Differential Privacy enables privacy and precise analytics. *Journal of Big Data* 10 (2023), 1–14. https://api.semanticscholar.org/CorpusID:259848534

[90] Haipei Sun, Kun Wu, Ting Wang, and Wendy Hui Wang. 2022. Towards Fair and Robust Classification. In *2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P)*. 356–376. doi:10.1109/EuroSP53844.2022.00030

[91] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. 2013. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199* (2013).
[92] Sebastian Szyller and N Asokan. 2023. Conflicting interactions among protection mechanisms for machine learning models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 15179–15187.
[93] Sebastian Szyller, Buse Gul Atli, Samuel Marchal, and N Asokan. 2021. Dawn: Dynamic adversarial watermarking of neural networks. In *Proceedings of the 29th ACM international conference on multimedia*. 4417–4425.
[94] The MinIO project. 2026. high performance data store for AI & Analytics minio_2026. https://www.min.io/
[95] Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. 2016. Stealing machine learning models via prediction APIs. In *25th USENIX security symposium (USENIX Security 16)*. 601–618.
[96] Cuong Tran, Keyu Zhu, Pascal Van Hentenryck, and Ferdinando Fioretto. 2024. On the effects of fairness to adversarial vulnerability. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. 521–529.
[97] Min-Jen Tsai, Ping-Yi Lin, and Ming-En Lee. 2023. Adversarial attacks on medical image classification. *Cancers* 15, 17 (2023), 4228.
[98] Yusuke Uchida, Yuki Nagai, Shigeyuki Sakazawa, and Shin'ichi Satoh. 2017. Embedding Watermarks into Deep Neural Networks. *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval* (2017). https://api.semanticscholar.org/CorpusID:13060737
[99] Daiju Ueda, Taichi Kakinuma, Shohei Fujita, Koji Kamagata, Yasutaka Fushimi, Rintaro Ito, Yusuke Matsui, Taiki Nozaki, Takeshi Nakaura, Noriyuki Fujima, et al. 2024. Fairness of artificial intelligence in healthcare: review and recommendations. *Japanese Journal of Radiology* 42, 1 (2024), 3–15.
[100] Shuyi Wang and Guido Zuccon. 2023. An analysis of untargeted poisoning attack and defense methods for federated online learning to rank systems. In *Proceedings of the 2023 ACM SIGIR International Conference on Theory of Information Retrieval*. 215–224.
[101] Dongxian Wu and Yisen Wang. 2021. Adversarial Neuron Pruning Purifies Backdoored Deep Models. *ArXiv* abs/2110.14430 (2021). https://api.semanticscholar.org/CorpusID:239998081
[102] Weilin Xu, David Evans, and Yanjun Qi. 2017. Feature squeezing: Detecting adversarial examples in deep neural networks. *arXiv preprint arXiv:1704.01155* (2017).
[103] Greg Yang, Tony Duan, J. Edward Hu, Hadi Salman, Ilya P. Razenshteyn, and Jungshian Li. 2020. Randomized Smoothing of All Shapes and Sizes. *ArXiv* abs/2002.08118 (2020). https://api.semanticscholar.org/CorpusID:211171876
[104] Yu Yang, Tian Yu Liu, and Baharan Mirzasoleiman. 2022. Not all poisons are created equal: Robust training against data poisoning. In *International Conference on Machine Learning*. PMLR, 25154–25165.
[105] Ruichen Yao, Ziteng Cui, Xiaoxiao Li, and Lin Gu. 2022. Improving fairness in image classification via sketching. *arXiv preprint arXiv:2211.00168* (2022).
[106] Dayong Ye, Sheng Shen, Tianqing Zhu, B. Liu, and Wanlei Zhou. 2022. One Parameter Defense—Defending Against Data Inference Attacks via Differential Privacy. *IEEE Transactions on Information Forensics and Security* 17 (2022), 1466–1480. https://api.semanticscholar.org/CorpusID:247447226
[107] Samuel Yeom, Irene Giacomelli, Matt Fredrikson, and Somesh Jha. 2017. Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting. *2018 IEEE 31st Computer Security Foundations Symposium (CSF)* (2017), 268–282. https://api.semanticscholar.org/CorpusID:2656445
[108] Muhammad Bilal Zafar, Isabel Valera, Manuel Gomez-Rodriguez, and Krishna P. Gummadi. 2015. Fairness Constraints: Mechanisms for Fair Classification. *ArXiv* abs/1507.05259 (2015). https://api.semanticscholar.org/CorpusID:8529258
[109] Richard S. Zemel, Ledell Yu Wu, Kevin Swersky, Toniann Pitassi, and Cynthia Dwork. 2013. Learning Fair Representations. In *International Conference on Machine Learning*. https://api.semanticscholar.org/CorpusID:490669
[110] Hongyang Zhang, Yaodong Yu, Jiantao Jiao, Eric P. Xing, Laurent El Ghaoui, and Michael I. Jordan. 2019. Theoretically Principled Trade-off between Robustness and Accuracy. *ArXiv* abs/1901.08573 (2019). https://api.semanticscholar.org/CorpusID:59222747
[111] Jialong Zhang, Zhongshu Gu, Jiyong Jang, Hui Wu, Marc Ph Stoecklin, Heqing Huang, and Ian Molloy. 2018. Protecting intellectual property of deep neural networks with watermarking. In *Proceedings of the 2018 on Asia conference on computer and communications security*. 159–172.
[112] Jinghuai Zhang, Hongbin Liu, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Data Poisoning based Backdoor Attacks to Contrastive Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 24357–24366.
[113] Yue Zhao and Maciej K. Hryniewicki. 2018. XGBOD: Improving Supervised Outlier Detection with Unsupervised Representation Learning. *2018 International Joint Conference on Neural Networks (IJCNN)* (2018), 1–8. https://api.semanticscholar.org/CorpusID:52988666
[114] Haizhong Zheng, Ziqi Zhang, Juncheng Gu, Honglak Lee, and Atul Prakash. 2019. Efficient Adversarial Training With Transferable Adversarial Examples. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2019), 1178–1187. https://api.semanticscholar.org/CorpusID:209501025
[115] Runkai Zheng, Rongjun Tang, Jianze Li, and Li Liu. 2022. Pre-activation distributions expose backdoor neurons. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1356, 14 pages.

Table 6: Trustworthy ML techniques and hyperparameters used

Defense Technique	Hyperparameter	Value
WM_{pre}	Watermarking fraction	0.1
AR_{in}	Trade-off regularization	6
AR_{post}	Weight decay regularization	e^{-9}
	Autoencoder noise-level	0.025
OR_{pre}	Contamination rate	0.1
	Number of estimators	20
OR_{in}	Forget rate	0.5
	Noise type	symmetric
OR_{post}	Prunning threshold	0.1
WM_{pre}	Trigger size	5x5
	Watermarking fraction	0.1
WM_{in}	Watermark strength	0.01
FP_{dep}	Num. of fingerprints	100
	Significance level (α)	0.01
DP_{in}	Maximum gradient norm	2.0
	Target epsilon (delta)	50 (e^{-5})
DP_{dep}	Target epsilon (delta)	50 (e^{-5})
EX_{dep}	Num. of SHAP samples	500

A Literature distribution

The Table below provides a breakdown of the venues where the work was selected.

Table 7: Selected venues and tally of work collected

Conference	Acronym	No.
Conf. on Neural Information Processing Sys.	NeurIPS	11
Int. Conf. on Machine Learning	ICML	09
Int. Conf. on Learning Representations	ICLR	08
Conf. on Comp. and Communications Security	CCS	03
Int. Conf. on Multimedia Retrieval	ICMR	03
Conf. on Comp. Vision and Pattern Recognition	CVPR	02
Int. Joint Conference on Artificial Intelligence	IJCAI	02
USENIX Security	Security	02
Arch. Supp. for Prog. Lang. and Oper. Systems	ASPLOS	01
Symp. on Security and Privacy	S&P	01
Netw. and Distrib. Syst. Security	NDSS	01
Winter Conf. on Appl. of Comp. Vision	WACV	01
Int. Joint Conf. on Neural Networks	IJCNN	01
Int. Conf. on Acoust., Speech, and Signal Process.	ICASSP	01
Assoc. for the Adv. of Artificial Intelligence	AAAI	01
J. of Big Data	JBD	01
Trans. on Inf. Forensics and Security	TIFS	01
Int. Conf. on Knowl. Discov. and Data Min.	KDD	01
Grey literature additions	-	04
Total	-	54

B Open Science

The tools utilized in this work are available in their respective repositories, and the datasets are well-known and easy to access through various platforms (e.g. huggingface). We make the Landseer code available at https://anonymous.4open.science/r/Landseer-FF4A for reviewers to peruse. If accepted, we aim to make the Landseer framework public under MIT License.

C Ethical Considerations

This work does not substantially affect the status quo of current practices in TML defenses. In particular, the defenses and datasets are public knowledge and are widely used in the field, and we do not foresee ethical effects of their use in this work. We expect that this work will foster the adoption and integration of ethically relevant tools to achieve fairness, privacy, and robustness.

D Generative AI Usage

System Implementation: We used GitHub Copilot to assist with the development of the Landseer framework and replicating some TML defenses. We manually verified the correctness of all the code generated by the tool. Our manual evaluation of Landseer and the respective defenses also serves to validate the correctness of the generated code.

Writing: We used Grammarly to help with editorial edits like grammar, spelling, and simple rephrasing. We verified all factual statements in the paper and ensured that no content generated by AI was included without verification.

Table 5: The table summarizes surveyed trustworthy ML techniques in Landseer, including the technique type, application stage, referenced work, publication year, key properties, and composability notes. Properties capture artifact availability (A) and whether it is official or third-party, reproducibility (R), replicability (P), self-contained (S), extensibility (E), modularity (M), and integration into Landseer (I). Symbols denote: ○ (absent), ●(present or official artifact), ◐(partially present or third-party artifact), and ✿ (integrated in Landseer). Properties S, E, and M are aggregated: ●if all are present, ◐if partially satisfied, and ○ if all are absent.

Type	Stage	Citation	Year	Properties (A,R,P,[S,E,M],I)	Non-Composability Reasons / Composability Notes
T^{ev}	Pre	Cisse et al. [21]	2017	○○○○	Not composable due to the absence of a publicly available artifact
	During	Madry et al. [59]	2017	●●●○	The framework is incompatible with the other tools selected for the experiment.
		Shafahi et al. [83]	2019	●●●○	The framework is incompatible with the other tools selected for the experiment.
		Zhang et al. [110] (AR_{in})	2019	●○●●✿	Not reproducible due to CUDA 9 dependency. Replicated via official artifact.
	Post	Papernot et al. [73]	2015	○○○○	Not composable due to the absence of a publicly available artifact
		Meng et al. [63] (AR_{post})	2017	●●●●✿	Containerized with minimal efforts
T^{ou}	Pre	Lai et al. [48]	2019	●●○○	Reproduced on official datasets but not replicated on CIFAR-10; not composable in our setup.
		Zhao et al. [113] (OR_{pre})	2018	●●●●✿	Added an adapter that runs outlier filtering and writes standardized output npy data artifacts for downstream tools.
		Borgnia et al. [11]	2020	○○○○	Not composable due to the absence of a publicly available artifact
	During	Han et al. [37] (OR_{in})	2018	●●●●✿	Changed python2 to python3
	Post	Li et al. [51] (OR_{post})	2023	●●●●✿	Containerized with minimal effort
T^{wm}	Pre	Sablayrolles et al. [82]	2020	●○○○	Irrecoverable issues with codebase and reproducing results
		Gu et al. [35] (WM_{pre})	2017	●●●◐✿	Reworked the data pipeline to pre-generate backdoored datasets, decoupling poisoning from training
	During	Adi et al. [2]	2018	●●○○	Challenging to containerize as it has an outdated dependency stack, including specific versions of PyTorch and CUDA.
		Rouhani et al. [81]	2019	○○○○	Not composable due to the absence of a publicly available artifact
		Uchida et al. [98] (WM_{in})	2017	●●●●✿	Changed Keras to Pytorch
		Zhang et al. [111]	2018	○○○○	Not composable due to the absence of a publicly available artifact
	Post	Chen et al. [17]	2018	○○○○	Not composable due to the absence of a publicly available artifact
	Deploy	Szyller et al. [93]	2021	●●●●✿	Changed pickle (.pkl) to Numpy .npy format and containerized
T^{fp}	During	Chen et al. [18]	2018	○○○○	Not composable due to the absence of a publicly available artifact
	Deploy	Lukas et al. [56]	2019	○○○○	Not composable due to the absence of a publicly available artifact
		Maini et al. [60] (FP_{dep})	2021	●●●●✿	Containerized with minimal effort
		Cao et al. [14]	2019	○○○○	Not composable due to the absence of a publicly available artifact
T^{dp}	Pre	Lin et al. [52]	2023	●○○○	Irrecoverable codebase error
		Canonne et al. [13]	2020	●●○○	Unable to replicate on CIFAR-10 due to limited pipeline documentation and integration guidance
		Subramanian et al. [89]	2023	○○○○	Not composable due to the absence of a publicly available artifact
	Post	Ye et al. [106]	2022	○○○○	Not composable due to the absence of a publicly available artifact
	During	Holohan et al. [41]	2019	●○●○	Unable to reproduce; CIFAR-10 replication <50% (CNN) and ~35% (ResNet), thus excluded.
	During , Deploy	Abadi et al. [1] (DP_{in}) (DP_{dep})	2016	●○●●✿	Unable to reproduce with ResNet18 (Opacus compatibility) but replicated with ResNet20 (no BatchNorm)
T^{gf}	Pre	Calmon et al. [27]	2017	●○○ ○	Not composable due to dataset incompatibility (tabular data)
		Zemel et al. [109]	2013	◐○○ ○	Not composable due to dataset incompatibility (tabular data)
	During	Zafar et al. [108]	2015	●○○ ○	Not composable due to dataset incompatibility (tabular data)
		Yao et al. [105] GF_{in}	2022	●●●◐✿	Modified dataset split to enable DP/DEO (original test set single-class); integrated sketch preprocessing as subprocess.
	Deploy	Hardt et al. [38]	2016	◐○○○	Not composable due to dataset incompatibility (tabular data)
T^{ex}	During	Alvarez-Melis et al. [4]	2018	○○○○	Not composable due to the absence of a publicly available artifact
	Deploy	Kim et al. [46]	2017	●●○○	Unable to replicate for CIFAR-10 due to missing required files
		Lundberg et al. [58] EX_{dep}	2017	●●●●✿	Applied SHAP to replicate the technique on CIFAR-10